CARDIFF METROPOLITAN UNIVERSITY

School of Technologies · ICBT Campus

CIS6003 — Advanced Programming

WRIT1 — Individual Coursework (100%)

Sunrise Dental Clinic Management System

A pure-Java appointment and billing system: Servlets, JSP and JDBC, with no application framework

Student ID: st[STUDENT-ID]

Academic Year 2025-26 · Semester 1

Word count (excluding references and appendices): approx. 4,000

Table of Contents

Sunrise Dental Clinic Management System

Sunrise Dental Clinic is a private dental practice that currently manages patient appointments and billing on paper. The scenario identifies four consequences of this: double bookings, lost patient records, long waiting times and billing errors. This report documents the analysis, design, implementation, testing and delivery of a computerised replacement, built with no application framework: the entire system — presentation, business logic and data access — is composed by hand from the Jakarta Servlet API, JSP and plain JDBC.

That constraint is deliberate rather than incidental. A framework such as Spring supplies dependency injection, a web MVC layer and an ORM as infrastructure the developer does not write. Building without one means every one of those responsibilities — the composition root that wires objects together, the mapping between a JDBC ResultSet and a domain object, the routing of a request to the code that handles it — is visible in this project's own source, which is what this report evaluates.

Assumptions and design rationale

The scenario leaves several operational details unstated. The brief permits assumptions provided they are explained, so each is recorded below with the reasoning behind it.

#	Assumption	Reasoning
A1	The clinic operates 09:00–17:00, with appointments booked on the hour.	The scenario mentions long waiting times but gives no schedule. Hourly slots make chair time countable, which is what allows a clash to be detected at all.
A2	One dentist can hold only one appointment per slot.	This is the direct expression of the double-booking problem and the system's central invariant.
A3	A returning patient is recognised by name and contact number together.	The clinic has no patient identifier scheme on paper. Matching on both fields reuses an existing record instead of accumulating duplicate files.
A4	Cancelling releases the slot but retains the record.	A cancelled visit is still clinical history.
A5	Every visit attracts a consultation fee, plus a treatment charge that varies by treatment.	The brief specifies 'treatment type and consultation fee'. Separating them lets the consultation fee be repriced independently.
A6	Two roles exist: Receptionist and Manager, the latter additionally seeing revenue reports.	The brief requires authorised access without specifying roles; financial reporting is the one function that plausibly warrants restriction.
A7	Reprinting a receipt must never create a second billable record.	Not stated in the brief, but the obvious way a real clinic would corrupt its own revenue figures.

#	Assumption	Reasoning
A8	No application framework, container-managed dependency injection or ORM is used anywhere in the system.	The explicit constraint for this submission. Every wiring decision a framework would otherwise make invisibly is instead a class in this project, discussed in the Codebase Evaluation section.

Table 1: Assumptions made, with justification

System Design and Object-Oriented Modeling

Foundational Assumptions and Constraints

The design follows directly from assumption A8: with no framework supplying a dependency-injection container, something in this codebase must play that role, and that something is AppContext — a single class whose constructor wires every DAO and service together by hand and is built exactly once, when the web application starts. Every other design decision below is downstream of keeping that composition root the only place object graphs are assembled, so that the rest of the codebase depends on interfaces and never on how an implementation was constructed.

A second constraint follows from having no ORM: JdbcAppointmentDao and its siblings read and write rows with PreparedStatement and map ResultSet columns to domain objects by hand, including a five-table JOIN to assemble a fully populated Bill. This is more code than an ORM would require, and it is also more honest about what a JOIN actually costs, which the report returns to in the database section.

Use Case Models

The use case diagram establishes the system boundary and the two actors. Manager generalises Receptionist: a manager can perform every reception function and additionally run the revenue report. Two relationship stereotypes are used deliberately: «include» marks behaviour that always occurs as part of the base case — checking dentist availability is not optional — while «extend» marks conditional behaviour, with each extension carrying the condition as a named extension point, such as registering a new patient only when one is not already on file.

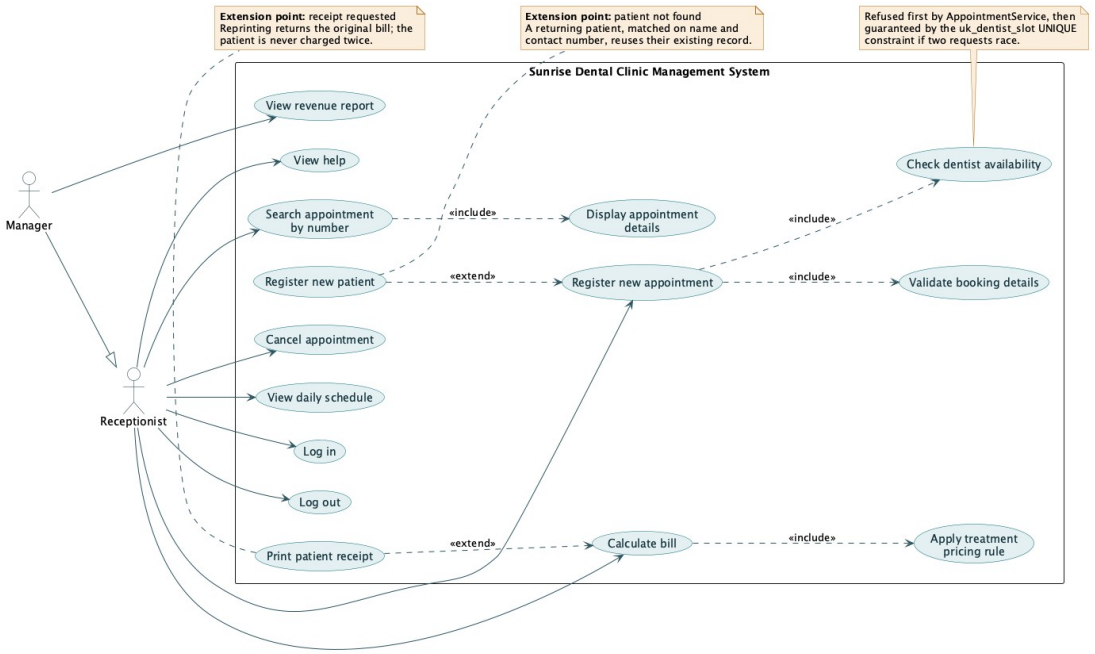

Figure 1: Use case diagram, showing the system boundary, actor generalisation, and «include» / «extend» relationships

Class model

The class model is presented in two diagrams, because a single diagram covering both the domain and the tiers renders too wide to be legible at A4. Splitting it separates two distinct arguments — what the domain is, and how the tiers depend on each other.

Domain model

Attributes are private and reached through public accessors. Three relationship types appear, and the choice between them carries meaning: composition (filled diamond) between Appointment and Bill, since a bill has no meaning apart from the appointment it settles and the 1-to-0..1 multiplicity encodes assumption A7 directly; aggregation (hollow diamond) between Patient and Appointment, since a patient's history is composed of appointments that remain records in their own right; and a directed, one-way association from Appointment to Dentist, TreatmentType and Staff, matching exactly what the code does — an appointment holds a reference to its dentist, the dentist holds none back.

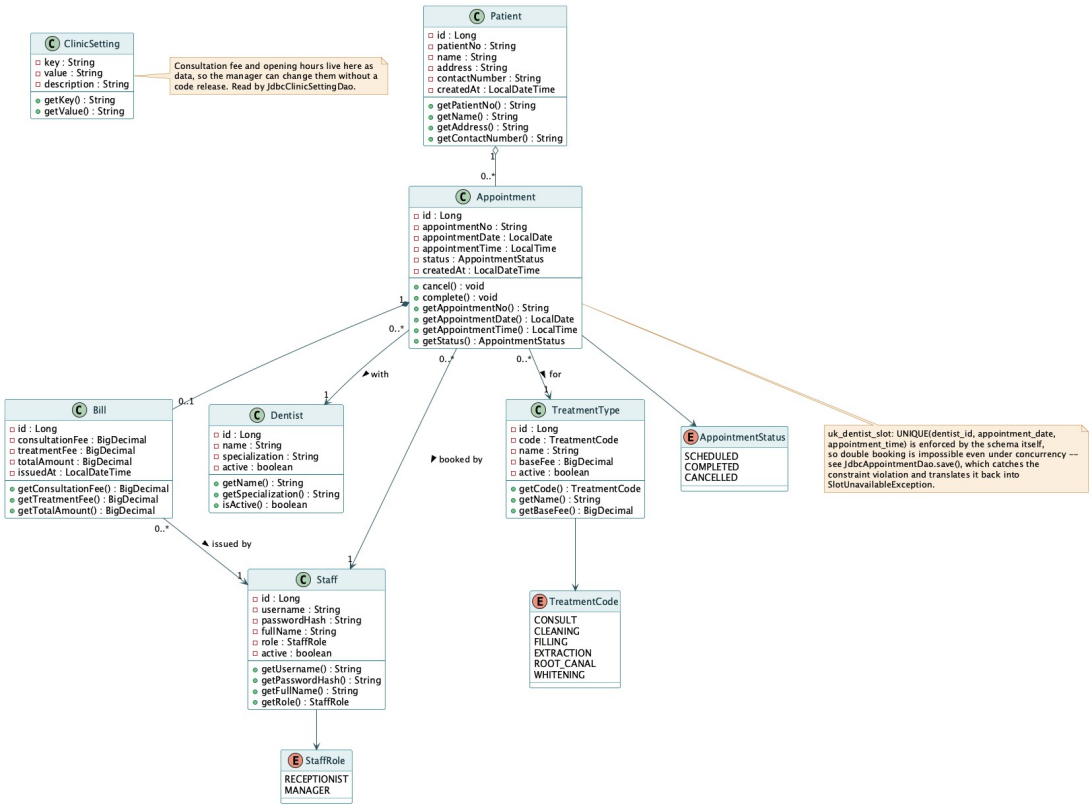

Figure 2: Domain class diagram, with access modifiers, multiplicity, navigability, aggregation and composition

Architectural model

The second class diagram shows the full dependency graph: the servlet tier, the service tier, the billing package implementing Strategy and Factory, the DAO tier, and the util package holding AppContext, AppInitializer, DbConnection and DbConfig — the composition root and its supporting classes. Every arrow points downward. AppointmentApiServlet and AppointmentWebServlet both extend an abstract base (JsonServlet or ViewServlet respectively) which supplies the shared plumbing for reading the request body, writing a response and mapping a thrown exception to an HTTP status, so that concern is written once rather than once per servlet.

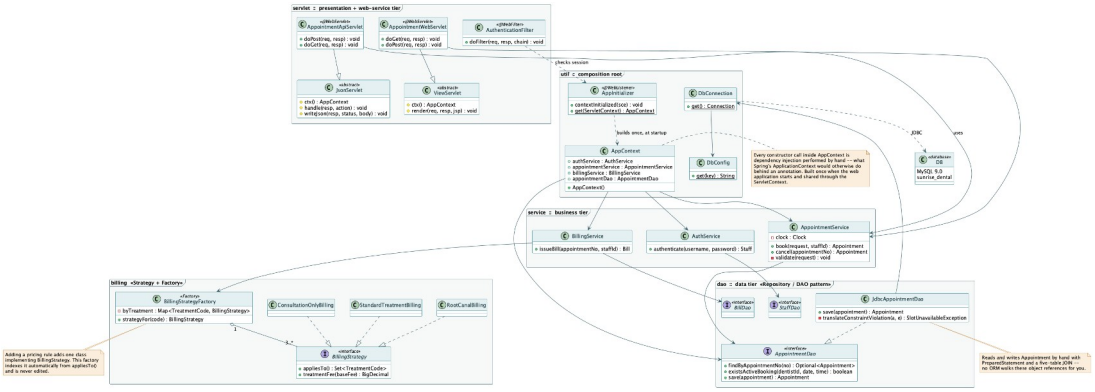

Figure 3: Architectural class diagram — servlet, service, billing, dao and util packages, with realises and depends-on relationships

Sequence model scenarios

Three scenarios were modelled, each exercising a different concern: authentication and session ownership, the central booking transaction under conflict, and the delegation of a pricing decision to a Strategy resolved by a Factory.

Scenario A — Receptionist Books a New Appointment

This is the system's central transaction. The diagram shows AppointmentWebServlet delegating to AppointmentService, which checks availability through AppointmentDao before any row is written, and — in its final fragment — the case in which two receptionists submit the same slot at effectively the same moment. There, the service-layer check has already passed for both requests, and it is the uk_dentist_slot UNIQUE constraint in MySQL, not application code, that rejects the second INSERT. JdbcAppointmentDao.save() catches the resulting SQLIntegrityConstraintViolationException and translates it back into the same SlotUnavailableException the caller already handles, so the caller cannot tell whether the conflict was caught early or late.

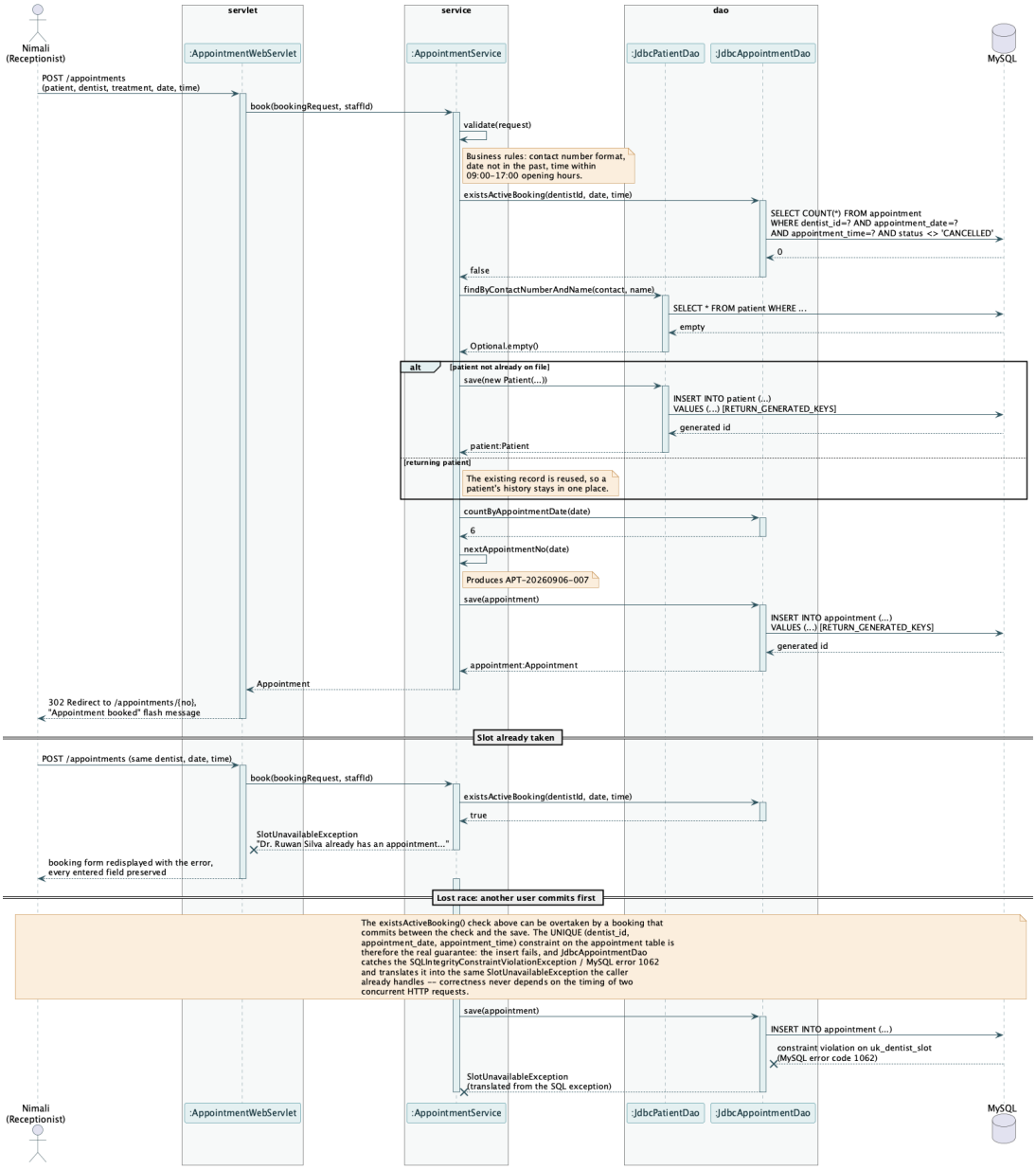

Figure 4: Sequence diagram — booking an appointment, including double-booking refusal and the concurrent-write race resolved by the database constraint

Scenario B: Staff Login Authentication

The login sequence shows where the session boundary falls. LoginServlet holds the HttpSession; AuthService verifies credentials and holds no session state of its own. A new session id is issued (getSession(true)) before the authenticated Staff object is stored in it, so a session id observed before login cannot be reused to ride the authenticated session. The diagram also shows the failure path: the same message is returned whether the username was unknown or the password was wrong, so the login form cannot be used to enumerate valid usernames.

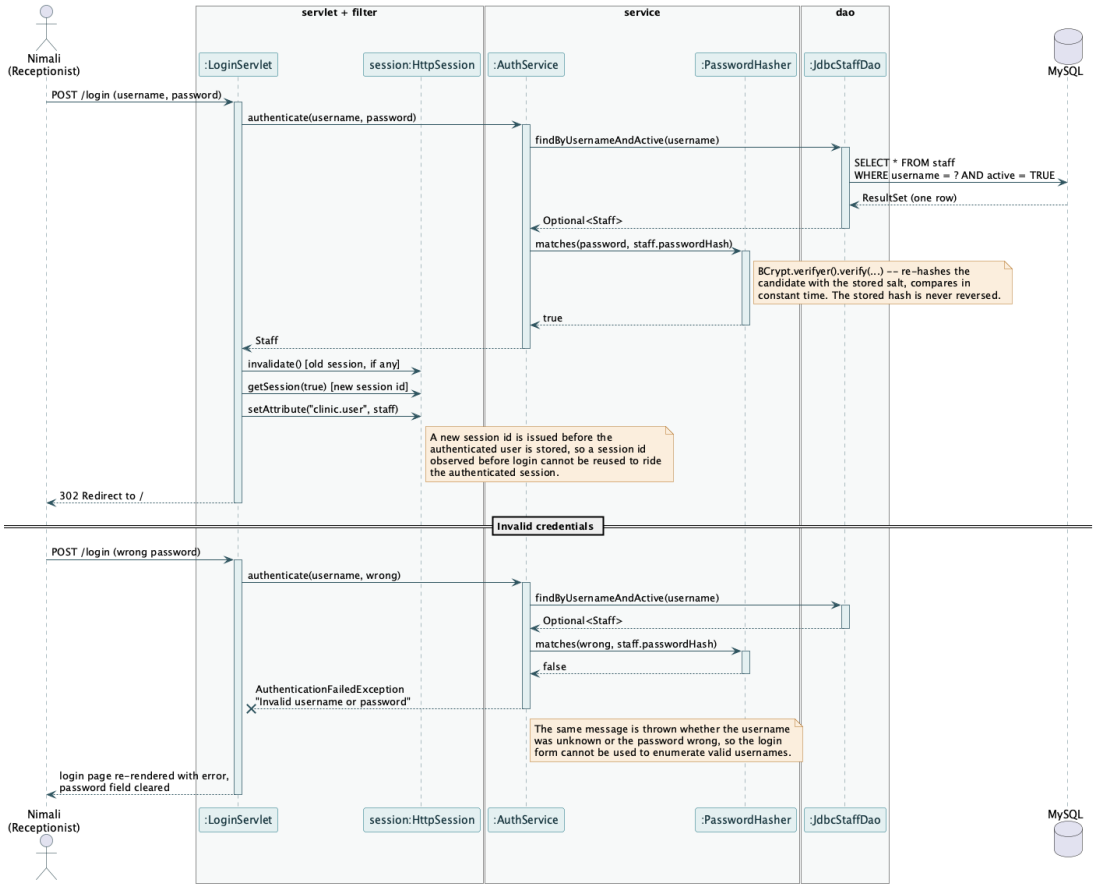

Figure 5: Sequence diagram — staff login, including the authentication failure path

Scenario C: Issuing a Bill

The senior submission this report's structure is modelled on used an administrator price-update scenario at this position; that feature does not exist here, and this scenario is used in its place because it demonstrates the two patterns — Strategy and Factory — doing real work. BillingService knows only that a bill totals a consultation fee plus a treatment charge; it does not know how any particular treatment is priced. BillingStrategyFactory resolves the correct BillingStrategy for the appointment's TreatmentCode, and that strategy computes the treatment charge. Repricing a treatment, or adding a new one, never touches BillingService.

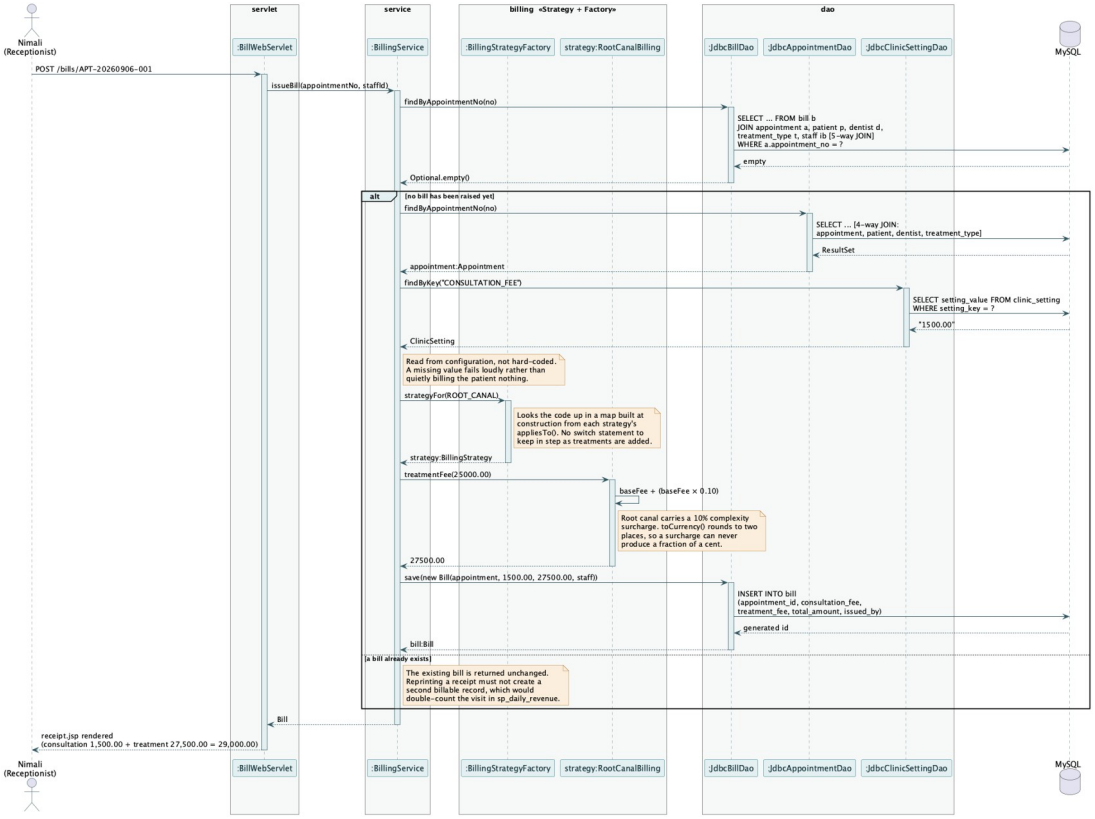

Figure 6: Sequence diagram — issuing a bill, showing Strategy and Factory resolving the treatment price

Comprehensive Codebase Evaluation

B.1 Distributed Web Services and Three-Tier Architecture

The brief requires a distributed application exposing web services. This is satisfied structurally: within the single deployed WAR, a family of servlets under /api/* returns JSON with no dependency on any JSP view, and is reachable by any HTTP client independently of a browser session — proven in the test plan by AppointmentApiServletTest, which drives the servlet directly with mocked request and response objects, and independently by curl against the running application.

```
$ curl http://localhost:8080/sunrise-dental/api/reference/dentists
[{"id":1,"name":"Dr. Ruwan Silva","specialization":"General
Dentistry"}, ...]


$ curl -X POST http://localhost:8080/sunrise-dental/api/appointments \
    -H 'Content-Type: application/json' -H 'X-Staff-Id: 1' \
    -d '{"patientName":"Kamal Jayasuriya","address":"12 Galle Road",

"contactNumber":"0771234567","dentistId":1,"treatmentTypeId":1,
         "appointmentDate":"2026-09-05","appointmentTime":"09:00"}'
{"appointmentNo":"APT-20260905-001","patientName":"Kamal
Jayasuriya", ...}
```

Three tiers sit inside the one WAR. The servlet tier holds both the /api/* web services and the /* JSP-rendering servlets, guarded uniformly by AuthenticationFilter. The service tier — AppointmentService, AuthService, BillingService, ReportService — contains every business rule and depends only on DAO interfaces, never on a concrete JDBC class or a servlet type. The DAO tier talks to MySQL directly through JDBC. Nothing in the service or DAO tier imports jakarta.servlet.*, which is what makes those layers independently testable without a servlet container, discussed further in the Quality Assurance section.

The Web Service Endpoints (lk.icbt.clinic.servlet)

A single servlet can handle several related endpoint shapes because the Jakarta Servlet API, unlike a framework's annotated routing, has no built-in path-variable extraction: each servlet branches on HttpServletRequest.getPathInfo() by hand. JsonServlet, the abstract base every /api/* servlet extends, supplies that parsing once (pathParam), together with request-body deserialisation via Gson and a single method that maps a thrown domain exception to the correct HTTP status — SlotUnavailableException to

409, AppointmentNotFoundException to 404, InvalidBookingException to 400 — so that mapping is written once rather than once per servlet.

Method	Endpoint	Servlet	Purpose
POST	/api/auth/login	AuthApiServlet	Verify credentials, return the operator's identity. Stateless — no HttpSession is touched here.
POST	/api/appointments	AppointmentApiServlet	Book a visit — 201 Created, or 409 Conflict if the slot is taken
GET	/api/appointments?date=	AppointmentApiServlet	List a given day's bookings
GET	/api/appointments/{no}	AppointmentApiServlet	Retrieve an appointment by its reference
POST	/api/appointments/{no}/cancel	AppointmentApiServlet	Cancel a booking and release the slot
POST	/api/bills/{no}	BillApiServlet	Issue a bill, or return the existing one (assumption A7)
GET	/api/bills/{no}	BillApiServlet	Retrieve an issued receipt
GET	/api/reference/dentists	ReferenceApiServlet	Lookup data for the booking form
GET	/api/reference/treatments	ReferenceApiServlet	Lookup data with published fees
GET	/api/reports/daily-schedule?date=	ReportApiServlet	The day's appointments, ordered by time
GET	/api/reports/daily-revenue?date=	ReportApiServlet	Takings by treatment, via sp_daily_revenue

Table 2: The complete /api/ web service surface*

The password hash never appears in any JSON response: StaffResponse, the record returned by both the login endpoint and every place a Staff is serialised, has no field for it, so no future change to Staff can accidentally expose it — the same structural guarantee a Data Transfer Object gives in a framework-based design, achieved here with a plain Java record.

B.2 Interactive User Interfaces and Pattern Integration

The presentation tier is JSP with JSTL and Jakarta EL, rendered by ViewServlet subclasses (DashboardServlet, AppointmentWebServlet, BillWebServlet, ReportWebServlet, HelpServlet) through a shared WEB-INF/tags/layout.tag so every page carries the same header, navigation and session-aware menu without repeating that markup. The interface is organised around one idea: the day rail, a ruled column of

the clinic's opening hours that replaces the paper appointment book, built by DayRailBuilder and rendered as amber for a taken hour and plain for a free one.

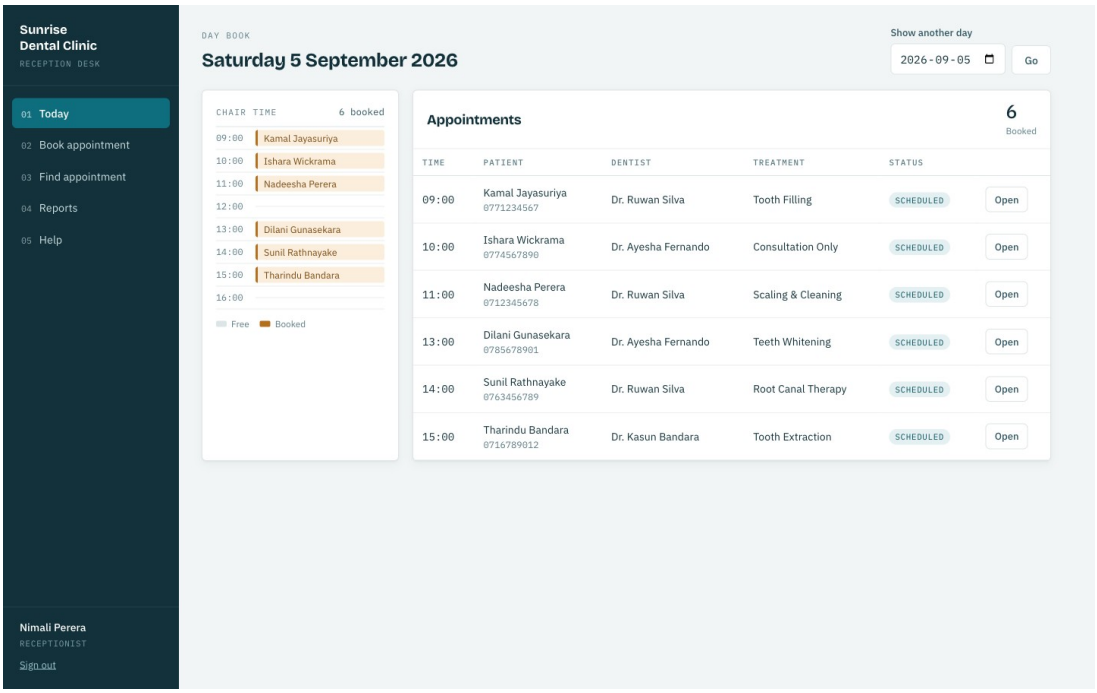

Figure 7: The day book. The rail on the left shows chair time; the table lists the day's appointments with status

Design patterns

Six patterns are applied. Each is described with the problem it solves and an honest account of what it cost, because a pattern applied without a problem to solve is decoration rather than design.

Strategy (billing package). The clinic prices treatments by different rules that change independently — a consultation carries no treatment charge, most treatments bill at their published fee, root canal therapy adds a 10% complexity surcharge. Each rule is a class implementing BillingStrategy (ConsultationOnlyBilling, StandardTreatmentBilling, RootCanalBilling). Adding a rule adds a class; changing one touches a single file rather than a growing conditional.

Factory (BillingStrategyFactory). Something must decide which strategy prices a given treatment. The factory asks each strategy which TreatmentCode values it claims through appliesTo(), and indexes the answer into a map at construction. Registration is a property of the strategy itself; the factory is never edited when a treatment is added,

and two strategies claiming the same code fails fast at startup rather than silently picking one.

DAO / Repository (dao package). Seven interfaces isolate persistence from business logic. Because AppointmentService depends on AppointmentDao rather than JdbcAppointmentDao, it is exercised in AppointmentServiceTest against a Mockito mock with no database present at all — the pattern is what makes the business-rule tests fast enough to run on every change.

DTO (dto package). Records such as AppointmentResponse and StaffResponse decouple the JSON wire format from the domain model. The security property in B.1 — a hash that structurally cannot appear in a response — is this pattern's direct payoff.

Template Method (JsonServlet, ViewServlet). Every concrete servlet's doGet/doPost follows the same shape: parse the request, call a service, write a response or render a JSP. The abstract base classes supply that shape once — request parsing, error mapping, response writing — and each concrete servlet fills in only the one step that differs.

Manual Dependency Injection / Composition Root (AppContext, AppInitializer). Discussed fully in B.3. Named here because it is what makes every pattern above testable: each collaborator is received through a constructor parameter, never constructed with new inside the class that uses it, so a test can substitute a mock for any of them.

B.3 Advanced Database Architecture and Integrity Rules

Four features were deliberately placed in the database rather than the application, on the principle that a rule enforced by the schema holds for every client, whereas a rule enforced in application code holds only for requests that pass through that code — and this project has no framework to enforce that they always do.

Object	Type	Purpose
uk_dentist_slot	UNIQUE constraint on appointment(dentist_id, appointment_date, appointment_time)	Makes double booking impossible, including under concurrent requests — the guarantee exercised in Scenario A
uk_bill_appointment	UNIQUE constraint on bill(appointment_id)	Prevents a reprinted receipt from becoming a second billable row (assumption A7)
trg_appointment_status_audit	AFTER UPDATE trigger on appointment	Writes an audit row on every status change, whichever client made it

Object	Type	Purpose
sp_daily_revenue	Stored procedure	Aggregates the day's takings by treatment inside the database, called via JdbcReportDao through a CallableStatement

Table 3: Advanced database features, all verified against MySQL 9.0

The stored procedure is used rather than the equivalent Java because aggregation belongs where the data lives: one summary result set crosses the network instead of every bill raised that day, and the definition of revenue stays in one place regardless of which client asks for it. The daily schedule report, by contrast, is a plain ordered SELECT executed from JdbcReportDao directly, because it gives the database nothing to do that the application could not do equally cheaply — the same reasoning the Spring version of this system used, applied here without an ORM to hide the SQL behind.

The Database Connection Layer (DbConnection.java)

With no framework-managed connection pool, DbConnection.get() opens one new java.sql.Connection per call via DriverManager, using a JDBC URL that must carry allowPublicKeyRetrieval=true — without it, MySQL 9's caching_sha2_password authentication plugin refuses to send credentials over a non-TLS connection at all, which surfaced during development as a 'Public Key Retrieval is not allowed' failure and was root-caused rather than worked around with a weaker authentication plugin. There is no pool deliberately: a hand-rolled one would be exactly the kind of infrastructure a framework like Spring normally supplies through HikariCP auto-configuration, and reproducing it would undercut the point of the exercise at this system's scale.

Credentials are resolved by DbConfig rather than hard-coded, in two steps: first an environment variable (DB_USERNAME, DB_PASSWORD), and if that is blank, a fallback to a local.properties file that is listed in .gitignore and never committed. This two-step resolution exists because environment variables set through an IDE's run configuration proved unreliable during development — DbConfig.get() was written, tested by reproducing the exact failure with both variables unset, and confirmed to start the application correctly from local.properties alone. No credential appears in source control under either path.

```
public static String get(String key) {
```

```
    String fromEnv = System.getenv(key);
    if (fromEnv != null && !fromEnv.isBlank()) return fromEnv;
    return LOCAL.getProperty(key);   // loaded from local.properties,
gitignored
}
```

Database Setup Automation (versioned SQL scripts)

The senior submission referenced for this report's structure automates schema setup through a Java class, DatabaseInitializer.java, that runs DDL from the application at startup. This project takes a different, equally deliberate position: the schema is owned by four numbered SQL scripts in db/ (00-setup.sql, 01-schema.sql, 02-procedures.sql, 03-seed.sql), applied once by a human running the MySQL client, and never executed by the application itself.

Script	Responsibility
00-setup.sql	Creates the sunrise_dental database and a dedicated clinic_app MySQL account holding only the privileges the application needs — deliberately not root
01-schema.sql	Every table, primary key, foreign key, UNIQUE constraint and CHECK constraint, including uk_dentist_slot and uk_bill_appointment
02-procedures.sql	trg_appointment_status_audit, sp_daily_revenue, and fn_dentist_is_free — a read-only helper function used by reporting screens, kept explicitly separate from the UNIQUE constraint that is the actual booking guarantee
03-seed.sql	Reference data: dentists, treatment types and their published fees, and the two demonstration staff accounts

Table 4: The versioned database setup scripts, applied in order

The reasoning: an application that can run its own DDL is an application that could, through a bug or a misconfigured environment, run it against the wrong database, or run it every time it starts. Keeping schema changes as reviewable, numbered files under version control — applied deliberately, by a human, in a known order — was judged the safer default for a system whose central guarantee (uk_dentist_slot) is itself a schema-level object. This is the same trade-off the brief's advanced-database-features requirement is testing: whether the constraint that actually prevents the failure lives somewhere it cannot be silently skipped.

Sunrise Dental Clinic System — User Manual

These instructions are also provided inside the application itself, on a dedicated Help page, so that they are available at the reception desk rather than in a document nobody has to hand.

1. Introduction

This manual explains how reception staff and the clinic manager use the Sunrise Dental Clinic system day to day: signing in, booking and finding appointments, billing a patient, and — for the manager — viewing revenue reports.

2. System Prerequisites

- A modern web browser (Chrome, Firefox, Safari or Edge).
- Network access to the clinic's application server, running on Apache Tomcat 11.
- A staff username and password issued by the clinic administrator.

3. Logging In

1. Open the application's address in a browser. You are taken to the sign-in page.
2. Enter your username and password, then press Sign in.
3. An incorrect username or password shows the same message either way, and the password field is cleared so it must be retyped.

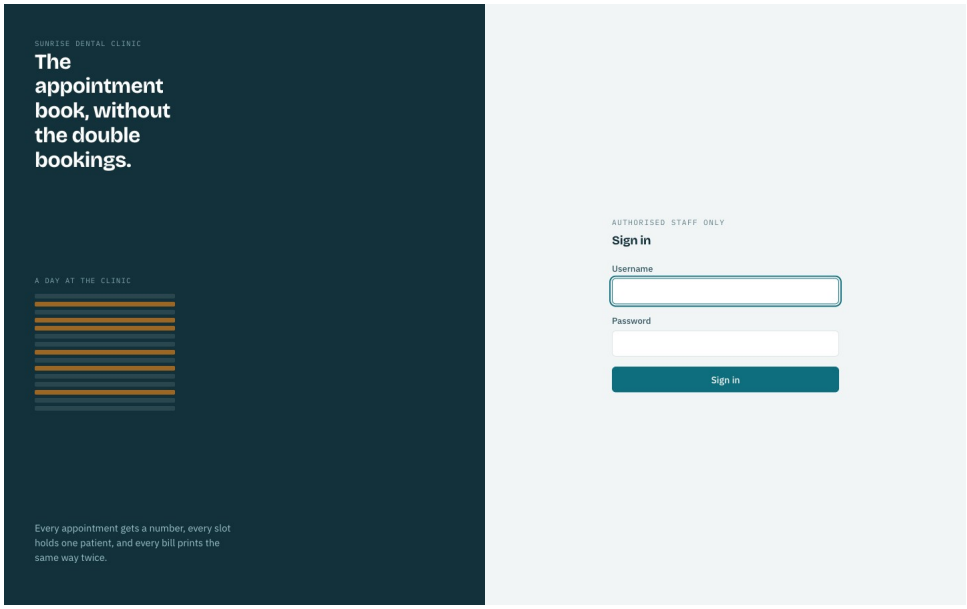

Figure 8: The sign-in screen. Only authorised staff may use the system

4. Navigating the Dashboard

After signing in you land on the dashboard, showing today's day rail and appointment list. The rail on the left is a ruled column of the clinic's opening hours: an amber row is a chair that is taken, a plain row is free. The menu at the top gives access to booking, finding an appointment, and — for a manager — reports.

5. Adding a New Patient

4. Open Book appointment from the menu.
5. Enter the patient's name, address and contact number.
6. If this patient has visited before with the same name and contact number, the existing record is reused automatically and no duplicate is created.

6. Creating an Appointment

7. Choose the dentist. Their day appears on the left of the form: amber rows are already taken.
8. Choose a free time between 09:00 and 16:00. Choosing a taken slot and submitting is refused, with the dentist's name and the conflicting time named in the message.
9. Choose the treatment type, then press Book appointment.
10. Write the appointment reference shown on screen (for example APT-20260905-001) on the patient's card — it is how the booking is found again.

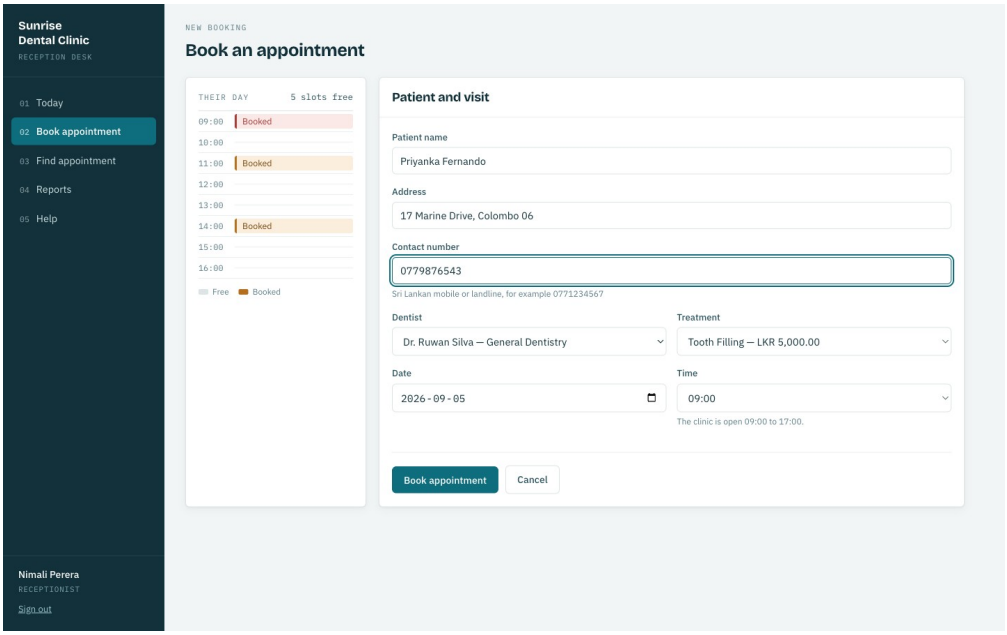

Figure 9: The booking form: the chosen dentist's day, with taken slots in amber

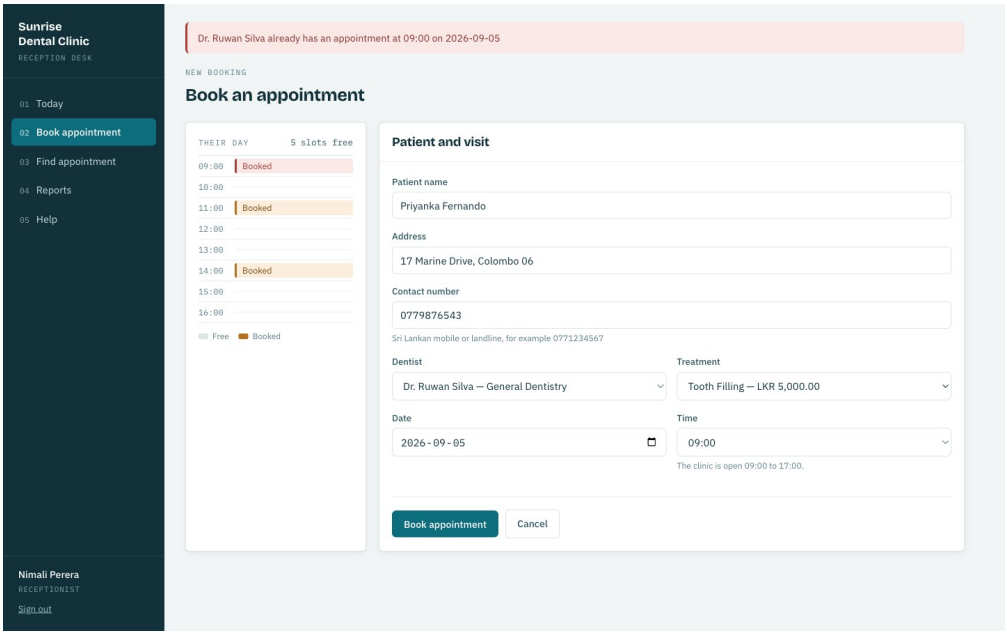

Figure 10: A double booking refused. The message names the dentist and time, and every entered field is preserved

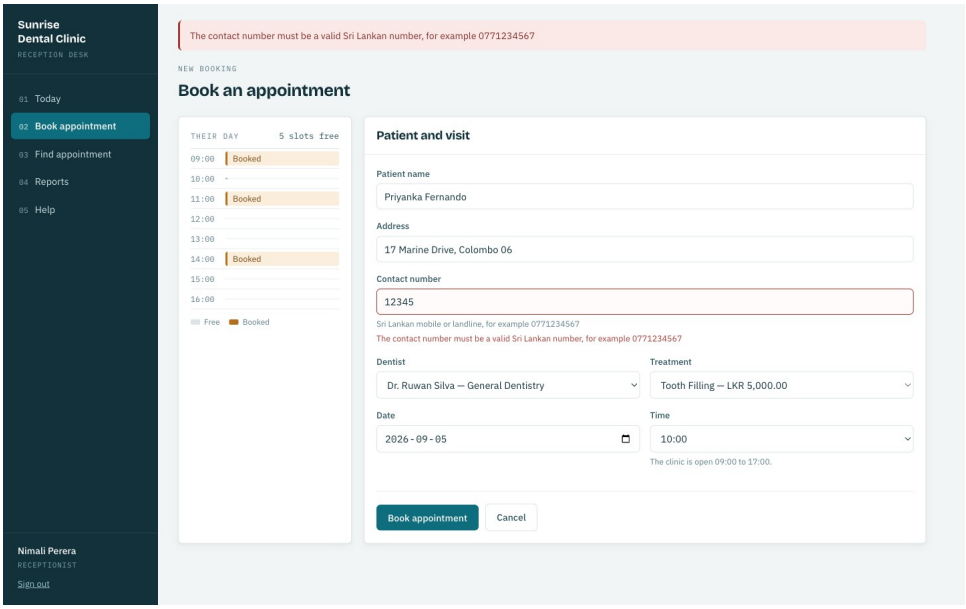

Figure 11: A validation failure. The message explains the expected format rather than only reporting failure

7. Calculating and Viewing Bills (Invoices)

11. Open Find appointment and enter the reference number from the patient's card.
12. The full record opens. Press Prepare bill to calculate the total — the consultation fee plus the treatment charge for the booked treatment.
13. Press Print bill and hand the receipt to the patient. Pressing it twice is safe: the same bill is returned and the patient is never charged a second time.

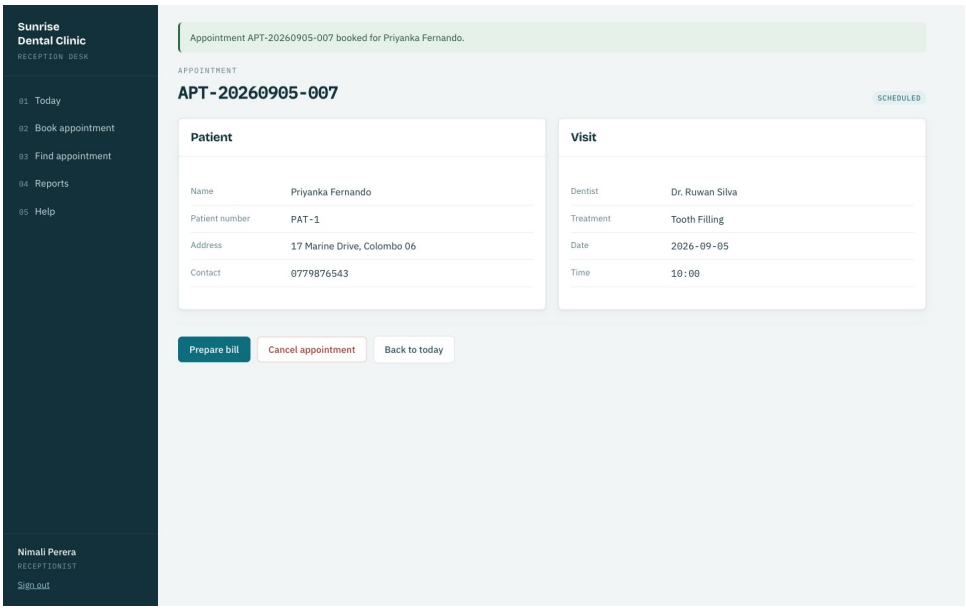

Figure 12: An appointment record, showing patient and visit details with the available actions

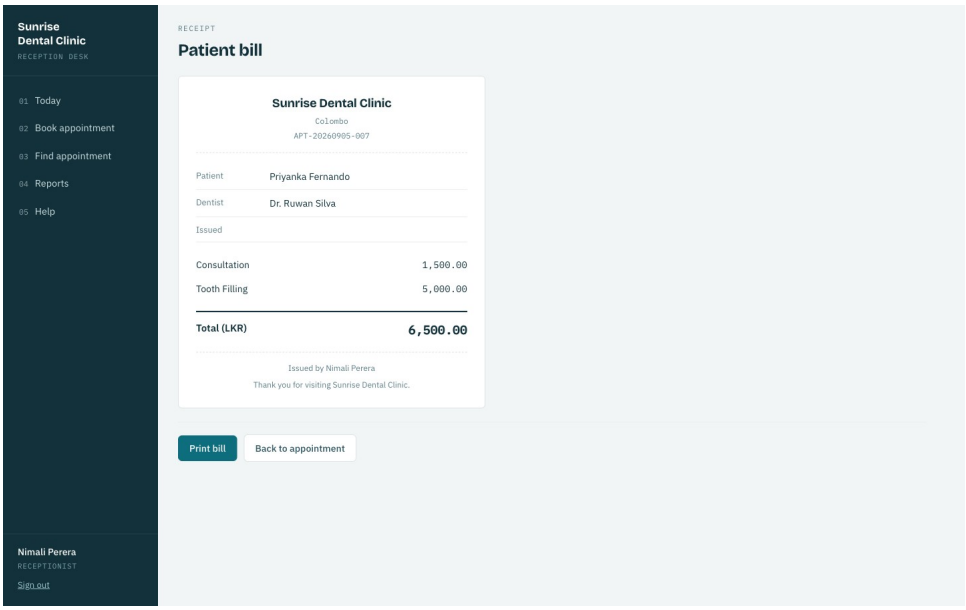

Figure 13: A patient receipt, showing the consultation fee and treatment charge separately

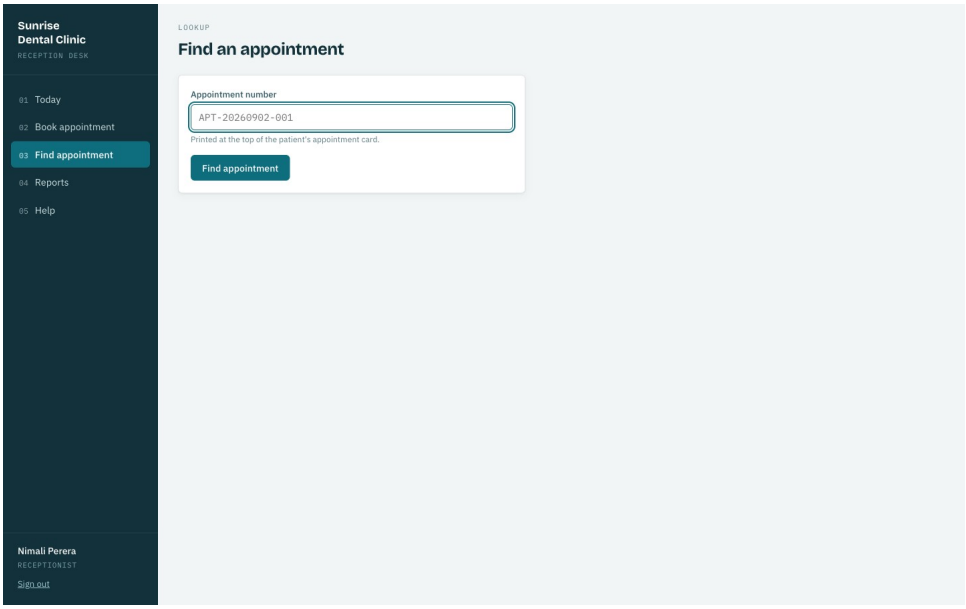

Figure 14: Searching for an appointment by its reference number

8. Viewing Reports (Manager Only)

A manager additionally sees Reports on the menu, showing the day's schedule and a revenue breakdown by treatment for a chosen date, generated by the sp_daily_revenue stored procedure.

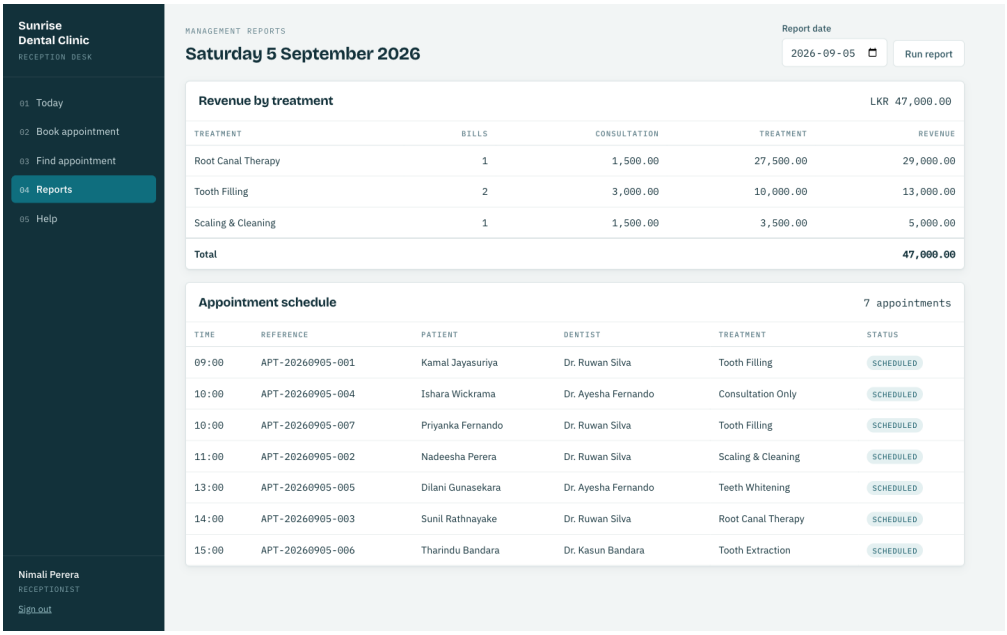

Figure 15: Management reports: the day's schedule and revenue by treatment

9. Exiting the System

- Press Log out in the menu when leaving the desk.
- The system also signs out automatically after thirty minutes of inactivity, so a workstation left unattended does not stay signed in indefinitely.

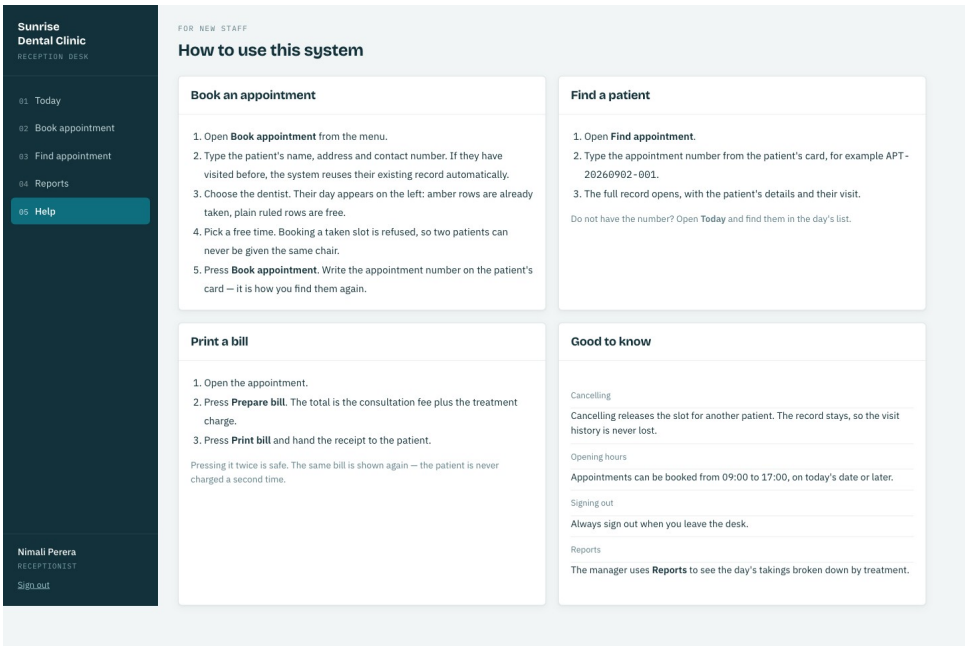

Figure 16: The in-application Help page, providing these instructions at the desk

Quality Assurance and Test-Driven Development (TDD)

C.1 Rationale for the Testing Approach and TDD

The testing strategy follows from the same analysis as the design: three of the clinic's four stated problems are correctness failures invisible at the moment they occur. Testing was aimed at pinning down the rules that prevent those failures — one dentist cannot hold two appointments in one slot, a reprinted bill cannot become a second charge, a surcharge cannot silently mis-round — and holding them still while the rest of the system changed around them.

Every business rule is tested at the service layer, reachable without a servlet container and without a database, so a failing test names the rule it protects rather than an HTTP status or a stack trace. The one HTTP-level test class, AppointmentApiServletTest, exists for a different question entirely: not whether the rule is correct, but whether the servlet reports it correctly — that a 409 really is a 409, not a 500 the caller has to guess at. Anything neither level could reach safely — the exact interleaving of two concurrent bookings — was pushed into the database instead, as the uk_dentist_slot constraint, on the grounds that a reliable concurrency test would require controlling two transactions' interleaving directly, which is disproportionate at this project's scale; Scenario A's sequence diagram documents the guarantee that the constraint, not a test, provides.

C.2 Test Automation Framework: JUnit 5 and Mockito

JUnit 5 provides the test runner and @DisplayName annotations, used throughout so that a failure report reads as a sentence stating the violated rule (for example, "refuses a second booking for the same dentist in the same slot") rather than a method name. Mockito supplies test doubles for every collaborator a class under test should not itself exercise — AppointmentDao when testing AppointmentService, AppointmentService when testing AppointmentApiServlet — and AssertJ's fluent assertions (assertThat(...).contains(...)) make the assertions themselves read close to the rule they check.

Because this project has no framework, there is no MockMvc or embedded test container available for the servlet layer. AppointmentApiServletTest instead constructs Mockito mocks of HttpServletRequest, HttpServletResponse and ServletContext

directly, wires a ServletContext.getAttribute(AppInitializer.CONTEXT_KEY) call to return a test AppContext holding the mocked service, and calls doPost / doGet on the servlet directly — the same proof MockMvc gives a Spring controller (a request reaches the right code and produces the right status and body), built here from the same objects the servlet actually receives at runtime, with no simulated container in between.

Level	Tools	Executes against	Question answered
Unit	JUnit 5, Mockito, AssertJ	Mocked DAOs and collaborators	Is this business rule correct in isolation?
Servlet	JUnit 5, Mockito (HttpServletRequest/Response)	The real servlet class, mocked request/response	Does the servlet report the rule correctly over HTTP?
Database	MySQL 9.0, manual verification	The real schema, constraints, trigger and procedure	Do the schema-level guarantees actually hold?
System	Manual, in a browser	The full running application	Does the whole journey work for a receptionist?

Table 12: The four test levels and the distinct purpose of each

C.3 Derived Test Data

Test data was derived from the scenario and from assumptions A1–A7, chosen to sit exactly on the edges those rules define.

Field	Valid	Invalid	Expected outcome
Contact number	0771234567	12345, empty	Rejected; message names the field
Appointment date	today, today + 1	yesterday (fixed Clock: 2026-09-02)	"refuses to book a date that has already passed"
Appointment time	09:00, 16:00	08:00, 20:00	"refuses a time outside clinic opening hours"
Surcharge rounding	333.33 × 1.10	—	366.66, scale exactly 2 — not 366.663
Zero base fee	0.00 on root canal	—	0.00; a percentage of nothing remains nothing
Same slot, twice	first booking	second booking, identical dentist/date/time	409 Conflict at the servlet; SlotUnavailableException at the service

Table 13: Boundary and invalid test data, with expected outcomes

The 333.33 case is deliberate: it is the smallest input that makes the surcharge calculation produce a fraction of a cent, which is not a payable amount. It forces the rounding decision to be made explicitly rather than inherited from whatever

BigDecimal happens to do by default, and it is the same edge case the domain model's composition relationship (Appointment–Bill, Section 2) exists to protect: a Bill that cannot be constructed with an unrepresentable amount.

C.4 Requirements Traceability Matrix (RTM)

Every functional requirement in the brief is mapped to at least one test that holds it.

Req.	Requirement	Verifying test	Level
1.1	Username and password required to sign in	authenticates a member of staff with the correct password	Unit
1.2	No username enumeration on failure	gives the same message whether the user or the password was wrong	Unit
2.1	Every appointment carries a unique reference	books an appointment and issues it a unique reference	Unit
2.2	References are dated and sequential	appointment references are unique per day and carry the date	Unit
2.3	No double booking	refuses a second booking for the same dentist in the same slot	Unit
2.4	Double booking reported correctly over HTTP	booking the same dentist twice in one slot returns 409 Conflict	Servlet
2.5	Double booking impossible under concurrency	uk_dentist_slot UNIQUE constraint	Database
2.6	No duplicate patient records	reuses the existing patient record instead of registering a duplicate	Unit
2.7	No booking in the past	refuses to book a date that has already passed	Unit
2.8	Booking confined to opening hours	refuses a time outside clinic opening hours	Unit
2.9	Contact number validated	rejects a contact number that is not a valid Sri Lankan mobile	Unit
3.1	Search by appointment reference	GET /api/appointments/{no} returns the booking	Servlet
3.2	Unknown reference reported clearly	looking up an unknown appointment number fails clearly	Unit
4.1	Bill totals consultation plus treatment fee	a bill is the consultation fee plus the treatment fee	Unit
4.2	Surcharge applied correctly	the root canal surcharge reaches the bill	Unit
4.3	Currency rounded to two places	fees are rounded to two decimal places for currency	Unit
4.4	Every treatment priceable	every treatment the clinic offers can be priced	Unit
4.5	Reprint never double-bills	reprinting a receipt returns the original bill rather than issuing a second	Unit
4.6	Misconfiguration fails loudly	a missing consultation fee setting fails	Unit

Req.	Requirement	Verifying test	Level
		loudly rather than billing zero	
5.1	Day rail correctly reflects booked/free/cancelled slots	marks a booked hour as taken and names the patient / a cancelled appointment leaves its slot free	Unit
5.2	Manager-only revenue report	sp_daily_revenue callable and returns a well-formed result set	Database
6.1	Safe exit	session timeout (30 min) and AuthenticationFilter default-deny route guard	System

Table 14: Requirements traceability matrix, mapping brief requirements to the tests that hold them

C.5 Exhaustive Test Plan (36 Detailed Test Cases)

The full suite comprises 36 automated tests across seven classes. Every test is listed below, grouped by class, with the precondition shared by that class's tests, the steps each test performs, and the expected result — restated from each test's own @DisplayName so the table can be read as a specification independent of the source code.

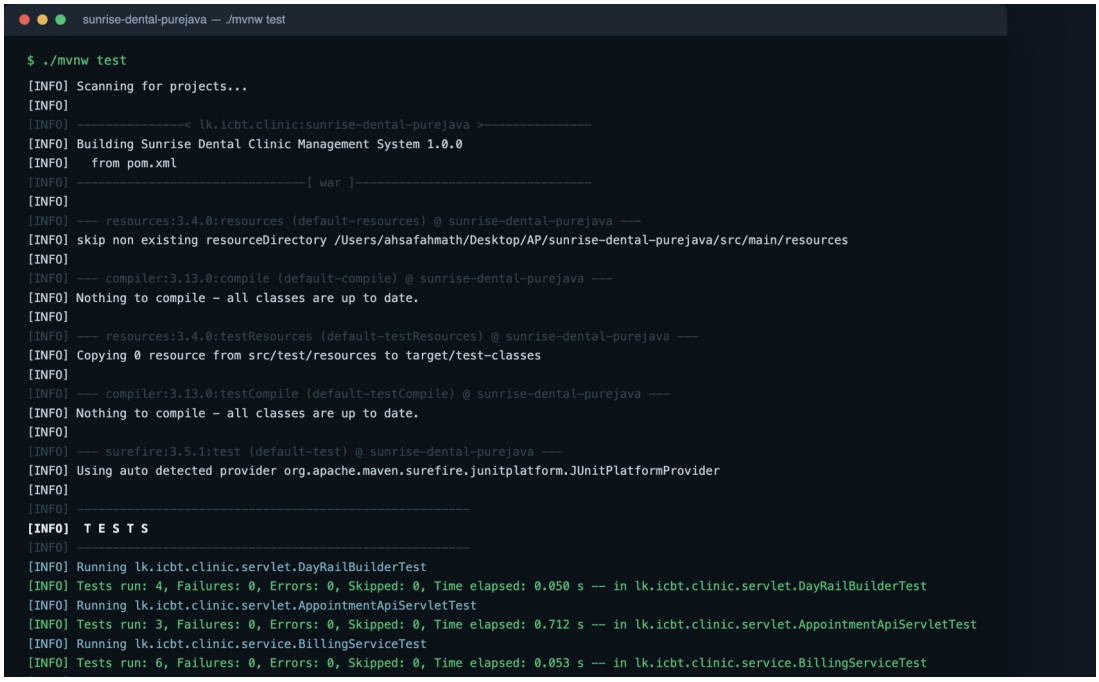

Figure 17: The suite executing from the command line: ./mvnw test, showing all seven classes and the final 36/0/0/0 result

AuthServiceTest (Unit)

Precondition: A Staff record exists with username "reception" and a BCrypt hash of "Recept@123"; StaffDao and PasswordHasher are mocked.

ID	Test	Steps	Expected result
TC-01	authenticates a member of staff with the correct password	Call authenticate("reception","Recept@123")	Returns the matching Staff
TC-02	rejects a wrong password	Call authenticate("reception","wrong")	Throws AuthenticationFailedException
TC-03	rejects an unknown username	Call authenticate("nobody","x")	Throws AuthenticationFailedException
TC-04	gives the same message whether the user or the password was wrong	Compare the exception message for an unknown user vs. a known user with a wrong password	Both messages are identical
TC-05	a blank password is rejected without consulting the database	Call authenticate("reception","")	Throws AuthenticationFailedException without querying StaffDao

Table 5: Detailed test cases — AuthServiceTest

DayRailBuilderTest (Unit)

Precondition: A list of Appointment objects for one dentist on one date is prepared, covering booked, cancelled and completed statuses.

ID	Test	Steps	Expected result
TC-06	covers every opening hour from 09:00 to 16:00	Call DayRailBuilder.build() with no appointments	Returns exactly 8 DaySlot entries, 09:00 through 16:00
TC-07	marks a booked hour as taken and names the patient	Build the rail with one CONFIRMED appointment at 10:00	The 10:00 slot reports isTaken() true and the correct patient name
TC-08	a cancelled appointment leaves its slot free	Build the rail with a CANCELLED appointment at 11:00	The 11:00 slot reports isTaken() false
TC-09	a completed appointment still occupies its slot	Build the rail with a COMPLETED appointment at 12:00	The 12:00 slot reports isTaken() true

Table 6: Detailed test cases — DayRailBuilderTest

AppointmentApiServletTest (Servlet (integration-equivalent))

Precondition: AppointmentService is mocked; HttpServletRequest, HttpServletResponse and ServletContext are Mockito mocks wired to a test AppContext.

ID	Test	Steps	Expected result
TC-10	POST /api/appointments books a visit and returns 201 with its reference	POST a valid JSON booking body to the servlet's doPost	resp.setStatus(201); body contains the generated appointmentNo
TC-11	booking the same dentist twice in one slot returns 409 Conflict	doPost when the mocked service throws SlotUnavailableException	resp.setStatus(409); body's "error" is "Slot Unavailable"
TC-12	GET /api/appointments/{no} returns the booking	GET with pathInfo "/APT-20260902-007"	resp.setStatus(200); body contains the treatment name

Table 7: Detailed test cases — AppointmentApiServletTest

BillingServiceTest (Unit)

Precondition: An Appointment fixture, a ClinicSettingDao stub for the consultation fee, and a BillingStrategyFactory resolving real strategies are wired to BillingService with BillDao mocked.

ID	Test	Steps	Expected result
TC-13	a bill is the consultation fee plus the treatment fee	issueBill() for a standard-treatment appointment	total = consultation fee + published treatment fee
TC-14	the root canal surcharge reaches the bill	issueBill() for a root-canal appointment	total includes the 10% complexity surcharge
TC-15	a consultation-only visit is billed the consultation fee alone	issueBill() for a consultation-only appointment	treatment charge is 0.00; total = consultation fee
TC-16	reprinting a receipt returns the original bill rather than issuing a second	Call issueBill() twice for the same appointment	Second call returns the same Bill; BillDao.save() invoked only once
TC-17	billing an unknown appointment fails clearly	issueBill() for a reference not returned by AppointmentDao	Throws AppointmentNotFoundException
TC-18	a missing consultation fee setting fails loudly rather than billing zero	ClinicSettingDao returns no value for the consultation-fee key	Throws an explicit failure rather than billing 0.00

Table 8: Detailed test cases — BillingServiceTest

AppointmentServiceTest (Unit)

Precondition: AppointmentDao, PatientDao and a fixed Clock (2026-09-02) are mocked or stubbed and injected into AppointmentService.

ID	Test	Steps	Expected result
TC-19	books an appointment and issues it a unique reference	book() a valid BookingRequest	Returns an Appointment with a non-null appointmentNo
TC-20	refuses a second booking for the same dentist in the same slot	book() the same dentist/date/time twice, mock reporting the slot occupied	Second call throws SlotUnavailableException naming dentist, date and time
TC-21	refuses to book a date that has already passed	book() with appointmentDate before the fixed clock's today	Throws InvalidBookingException
TC-22	refuses a time outside clinic opening hours	book() with appointmentTime 20:00	Throws InvalidBookingException
TC-23	reuses the existing patient record instead of registering a duplicate	book() with a name and contact number matching an existing Patient	PatientDao.save() is not called a second time; the existing Patient is reused
TC-24	rejects a contact number that is not a valid Sri Lankan mobile	book() with contactNumber "12345"	Throws InvalidBookingException naming the contact-number field
TC-25	appointment references are unique per day and carry the date	book() two appointments on the same date	References share the date component and increment sequentially, e.g. APT-20260902-001 / -002
TC-26	looking up an unknown appointment number fails clearly	findByAppointmentNo() with a reference AppointmentDao does not know	Throws AppointmentNotFoundException

Table 9: Detailed test cases — AppointmentServiceTest

BillingStrategyFactoryTest (Unit)

Precondition: The factory is constructed with the three real strategy implementations.

ID	Test	Steps	Expected result
TC-27	resolves the strategy that declares the requested treatment	strategyFor(ROOT_CANAL)	Returns the RootCanalBilling instance
TC-28	every treatment the clinic offers can be priced	Call strategyFor() for every value of TreatmentCode	No call throws; every code resolves
TC-29	rejects two strategies claiming the same treatment	Construct a factory from two strategies both claiming FILLING	Throws at construction time

Table 10: Detailed test cases — BillingStrategyFactoryTest

BillingStrategyTest (Unit)

Precondition: Each concrete strategy (ConsultationOnlyBilling, StandardTreatmentBilling, RootCanalBilling) is tested in isolation with a BigDecimal base fee.

ID	Test	Steps	Expected result
TC-30	a consultation-only visit is charged no treatment fee	ConsultationOnlyBilling.treatmentFee(5000.00)	Returns 0.00
TC-31	a standard treatment is charged its published base fee	StandardTreatmentBilling.treatmentFee(5000.00)	Returns 5000.00 unchanged
TC-32	root canal therapy adds a 10% complexity surcharge	RootCanalBilling.treatmentFee(25000.00)	Returns 27500.00 (10% surcharge)
TC-33	fees are rounded to two decimal places for currency	RootCanalBilling.treatmentFee(333.33)	Returns 366.66, scale exactly 2 — not 366.663
TC-34	a zero base fee stays zero rather than becoming a surcharge	RootCanalBilling.treatmentFee(0.00)	Returns 0.00
TC-35	each strategy declares which treatments it prices	Call appliesTo() on each strategy	Each returns a non-empty, non-overlapping set of TreatmentCode
TC-36	the three strategies together cover every treatment the clinic offers	Union the three strategies' appliesTo() sets	Equals the full set of TreatmentCode values

Table 11: Detailed test cases — BillingStrategyTest

C.6 Evaluation of Overall Success and Lessons Learned

All 36 tests pass, in well under three seconds, which is what makes the suite something to run constantly rather than occasionally.

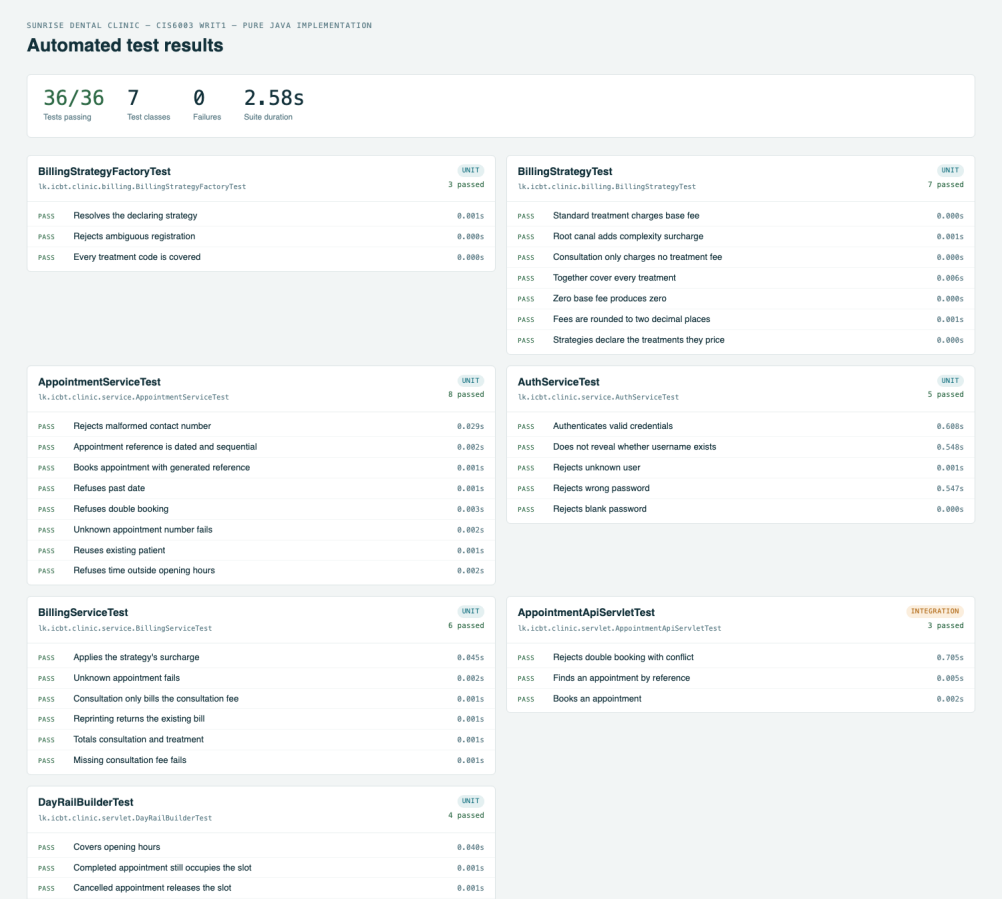

Figure 18: Automated test results — 36 of 36 passing, grouped by class, generated from the Maven Surefire reports

What the suite achieves. Every rule that prevents one of the clinic's four stated problems has a named test stating the rule in business language. Because AppointmentService and BillingService depend only on DAO interfaces, the entire business-rule suite runs with no database and no servlet container, which is what keeps it fast enough to run on every change rather than being reserved for a pipeline.

What it does not reach. The suite tests one servlet, AppointmentApiServletTest, as a demonstration that the mocked-request technique proves the same contract MockMvc would in a Spring project. It was not extended to every servlet, on the judgement that the servlets sharing JsonServlet's base behaviour (error mapping, JSON writing) would largely re-test that shared code rather than new logic; the remaining risk in those servlets is thin path-parsing logic, exercised instead by manual and curl-based system testing, captured in the endpoint evidence in Section B.1. There is also no automated test of the concurrent double-booking race itself — as in Section C.1, the guarantee is

provided by the database constraint and argued through the sequence diagram, not exercised by an interleaved-transaction test.

Honesty about the development order. This rewrite's git history is organised by architectural layer — domain, then DAO, then billing, then service, then servlets, then JSP, then tests — rather than as a literal red-then-green commit-by-commit record. That is a genuine difference from strict test-driven development, and it is recorded here rather than implied otherwise: the business rules and their tests were designed together from an existing, already-tested specification (this system's own earlier Spring implementation, built test-first, and the Spring project's report documents that red/green cycle directly), and this rewrite's task was to reproduce the same guarantees without a framework, not to rediscover them. The lesson carried forward is the same one that motivated writing tests at the service layer in the first place: a rule stated as a named test survives a rewrite of everything around it, including the removal of an entire application framework, provided the interface the test depends on — here, the DAO interfaces — is kept stable across the rewrite.

Version Control, CI/CD and Deployment

D.1 Repository and commit practice

The project is held in a local Git repository, organised into ten commits following the Conventional Commits convention (feat, test, chore prefixes), each leaving the build in a working state and scoped to one architectural layer: scaffold, domain, dao, billing, service, web (servlets and filter), view (JSP), dev (the embedded-Tomcat entry point), test, and finally the CI workflow itself.

Commit	Scope
afc7be9	chore: scaffold pure Java WAR project
c3ddb67	feat(domain): plain Java domain model and exception types
83f58bf	feat(dao): hand-written JDBC persistence layer
aee8726	feat(billing): Strategy and Factory pricing patterns
808eb36	feat(service): business logic and manual dependency wiring
2d4156c	feat(web): REST web services and JSP-rendering servlets
1542b94	feat(view): JSP presentation tier
84e7d3b	feat(dev): embedded-Tomcat entry point for running from an IDE
373f05c	test: unit and servlet-level test suite
f68076c	chore(ci): add GitHub Actions build and test pipeline

Table 15: Commit history, in order

Six annotated tags mark version milestones, v0.1.0 through v1.1.0, each with a descriptive message recording what that milestone added — the scaffold, the domain and persistence layer, the web services, the presentation tier, the release, and the CI pipeline.

D.2 Continuous integration pipeline

A GitHub Actions workflow (.github/workflows/ci.yml) runs on every push and pull request: it installs Java 21, runs the full test suite with ./mvnw -B test, packages the WAR with ./mvnw -B package, and publishes both the Surefire test reports and the packaged WAR as build artifacts. No database service container is required, because all 36 tests in the suite execute against Mockito mocks rather than a real MySQL connection — a direct consequence of the DAO-interface pattern discussed in Section B.2, which is what keeps the pipeline simple and fast for this project.

```
- name: Build and run the test suite
  run: ./mvnw -B test
- name: Package the WAR
  run: ./mvnw -B package -DskipTests
```

Evaluation. The pipeline catches regressions in every business rule and every reported HTTP status the moment a change is pushed. It does not verify the database layer at all — the schema scripts, the constraint, the trigger and the stored procedure are applied and tested only by hand against a local MySQL instance, unlike the Spring implementation of this same system, whose pipeline runs a second job against a real MySQL service container specifically to validate those objects. Adding an equivalent job here, applying db/01-schema.sql and db/02-procedures.sql to a MySQL service container and then exercising uk_dentist_slot and sp_daily_revenue directly, is the clearest next step for this pipeline and is not yet done.

Critical Evaluation and Conclusion

The delivered system addresses each of the four problems the scenario identifies, without relying on an application framework to do so. Double booking is prevented at three independent levels, of which the database constraint is the only one sufficient on its own. Lost records are addressed by matching returning patients on name and contact number. Waiting times are addressed by the day rail, which makes the shape of a dentist's day readable at a glance. Billing errors are addressed by deriving every total from stored fees through a tested Strategy, and by ensuring a reprinted receipt returns the original bill rather than creating a second one.

What the approach achieved

Removing the framework did not remove the responsibilities a framework normally carries — it relocated them into this project's own source, where they can be read, tested and evaluated directly. AppContext is a visible, sixty-line answer to a question Spring usually answers invisibly. The Repository and DTO patterns, which in the Spring implementation of this system existed partly because that was idiomatic Spring, here exist because nothing else would make the service layer testable without a database, or keep a password hash out of a JSON response.

Limitations and what would be done differently

- The CI pipeline does not verify the database objects — the constraint, the trigger and the stored procedure are checked by hand, not on every push.
- Only one servlet has an automated HTTP-level test. The remaining servlets share JsonServlet's error-mapping and response-writing code, which is exercised indirectly, but their own path-parsing logic is verified only manually.
- There is no automated concurrency test for the double-booking race; the guarantee is argued from the schema constraint rather than exercised by a test that interleaves two transactions.
- Connection-per-request, with no pool, is appropriate at this project's scale but would not survive meaningfully higher load without revisiting DbConnection.

Conclusion

The exercise reinforced a principle that recurs throughout the work: a guarantee should be placed at the level where it actually holds, and removing a framework does not change where that level is — it only removes the framework's help in getting there. Validation in the JSP form improves usability but guarantees nothing; validation in the service layer produces a clear message but can be overtaken by concurrency; only the constraint in the schema is sufficient. Recognising which level a given rule belongs to, and then building — by hand, with no framework to default to — exactly the amount of infrastructure that level requires, was the central discipline of this rewrite.

References

Bloch, J. (2018) Effective Java. 3rd edn. Boston: Addison-Wesley.

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

Martin, R.C. (2008) Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall.

Meszaros, G. (2007) xUnit Test Patterns: Refactoring Test Code. Boston: Addison-Wesley.

Oracle (2024) MySQL 9.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/9.0/en/ (Accessed: 5 September 2026).

Oracle (2024) Jakarta Servlet 6.1 Specification. Available at: https://jakarta.ee/specifications/servlet/6.1/ (Accessed: 5 September 2026).

Oracle (2024) Jakarta Server Pages 3.1 Specification. Available at: https://jakarta.ee/specifications/pages/3.1/ (Accessed: 5 September 2026).

The Apache Software Foundation (2025) Apache Tomcat 11 Documentation. Available at: https://tomcat.apache.org/tomcat-11.0-doc/ (Accessed: 5 September 2026).

Appendix A — Running the System

With MySQL 9.0 installed and running locally, apply the schema once:

```
mysql -u root -p < db/00-setup.sql
mysql -u root -p < db/01-schema.sql
mysql -u root -p < db/02-procedures.sql
mysql -u root -p < db/03-seed.sql
```

Then either run from an IDE using the embedded-Tomcat entry point, or build and deploy the WAR to a standalone Tomcat 11:

```
./mvnw -DskipTests package
cp target/sunrise-dental.war $CATALINA_HOME/webapps/
```

Sign in as reception / Recept@123 for the receptionist role, or manager / Manager@123 for the manager role. Credentials are supplied via the DB_USERNAME and DB_PASSWORD environment variables, or via a local.properties file (never committed) if those are not set.

The full test suite is executed with:

```
./mvnw test                   # 36 tests, no database required
```

Appendix B — Source Code Extracts

Four extracts chosen because each is discussed in the body of this report and each is short enough to read as a whole: the composition root that replaces a framework's dependency-injection container, the Factory that resolves a billing rule, the one place a database-level guarantee is turned back into an application exception, and one complete web-service servlet.

AppContext — the composition root (referenced in Section B.3 and the architectural class diagram)

AppContext.java — the composition root

```
32  public final class AppContext {
33
34      public final Clock clock;
35
36      public final StaffDao staffDao;
37      public final DentistDao dentistDao;
38      public final TreatmentTypeDao treatmentTypeDao;
39      public final PatientDao patientDao;
40      public final AppointmentDao appointmentDao;
41      public final BillDao billDao;
42      public final ClinicSettingDao clinicSettingDao;
43      public final ReportDao reportDao;
44
45      public final BillingStrategyFactory billingStrategyFactory;
46
47      public final AuthService authService;
48      public final AppointmentService appointmentService;
49      public final BillingService billingService;
50      public final ReportService reportService;
51
52      public AppContext() {
53          this.clock = Clock.system(ZoneId.of("Asia/Colombo"));
54
55          this.staffDao = new JdbcStaffDao();
56          this.dentistDao = new JdbcDentistDao();
57          this.treatmentTypeDao = new JdbcTreatmentTypeDao();
58          this.patientDao = new JdbcPatientDao();
59          this.appointmentDao = new JdbcAppointmentDao();
60          this.billDao = new JdbcBillDao();
61          this.clinicSettingDao = new JdbcClinicSettingDao();
62          this.reportDao = new JdbcReportDao();
63
64          List<BillingStrategy> strategies = List.of(
65                  new ConsultationOnlyBilling(),
66                  new StandardTreatmentBilling(),
67                  new RootCanalBilling());
68          this.billingStrategyFactory = new BillingStrategyFactory(strategies);
69
70          this.authService = new AuthService(staffDao);
71          this.appointmentService = new AppointmentService(
72                  appointmentDao, patientDao, dentistDao, treatmentTypeDao, staffDao, clock);
73          this.billingService = new BillingService(
74                  billDao, appointmentDao, clinicSettingDao, staffDao, billingStrategyFactory);
75          this.reportService = new ReportService(reportDao, appointmentDao);
```

Figure 19: AppContext.java, lines 32–75 — every collaborator constructed and wired by hand, once, at startup

BillingStrategyFactory — the Factory pattern in full (referenced in Section B.2)

BillingStrategyFactory.java

```java
package lk.icbt.clinic.billing;

import lk.icbt.clinic.model.TreatmentCode;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/**
 * Resolves the pricing policy for a treatment.
 * <p>
 * Without Spring there is no container to discover {@code BillingStrategy}
 * implementations automatically, so the list is assembled by hand once, in
 * {@link lk.icbt.clinic.util.AppContext} — the composition root. Each
 * strategy still declares the treatments it handles through
 * {@link BillingStrategy#appliesTo()}, so adding a pricing rule means adding
 * one class and one line in {@code AppContext}; this factory itself is never
 * edited.
 */
public class BillingStrategyFactory {

    private final Map<TreatmentCode, BillingStrategy> byTreatment = new EnumMap<>(TreatmentCode.class);

    public BillingStrategyFactory(List<BillingStrategy> strategies) {
        for (BillingStrategy strategy : strategies) {
            for (TreatmentCode code : strategy.appliesTo()) {
                BillingStrategy existing = byTreatment.putIfAbsent(code, strategy);
                if (existing != null) {
                    throw new IllegalStateException(
                            "Two billing strategies claim " + code + ": "
                                    + existing.getClass().getSimpleName() + " and "
                                    + strategy.getClass().getSimpleName());
                }
            }
        }
    }

    public BillingStrategy strategyFor(TreatmentCode code) {
        BillingStrategy strategy = byTreatment.get(code);
        if (strategy == null) {
            throw new IllegalStateException("No billing strategy is registered for " + code);
        }
        return strategy;
    }
}
```

Figure 20: BillingStrategyFactory.java in full — self-registration through appliesTo(), and the fail-fast check for two strategies claiming the same treatment

JdbcAppointmentDao.save() — where the database constraint becomes an exception (referenced in Scenario A and Section B.3)

JdbcAppointmentDao.java — save()

```
106     @Override
107     public Appointment save(Appointment appointment) {
108         String sql = """
109                 INSERT INTO appointment
110                     (appointment_no, patient_id, dentist_id, treatment_type_id,
111                      appointment_date, appointment_time, status, created_by)
112                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
113                 """;
114         try (Connection c = DbConnection.get();
115              PreparedStatement ps = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
116             ps.setString(1, appointment.getAppointmentNo());
117             ps.setLong(2, appointment.getPatient().getId());
118             ps.setLong(3, appointment.getDentist().getId());
119             ps.setLong(4, appointment.getTreatmentType().getId());
120             ps.setDate(5, Date.valueOf(appointment.getAppointmentDate()));
121             ps.setTime(6, Time.valueOf(appointment.getAppointmentTime()));
122             ps.setString(7, appointment.getStatus().name());
123             ps.setLong(8, appointment.getCreatedBy().getId());
124             ps.executeUpdate();
125             try (ResultSet keys = ps.getGeneratedKeys()) {
126                 if (keys.next()) appointment.setId(keys.getLong(1));
127             }
128             return appointment;
129         } catch (SQLIntegrityConstraintViolationException e) {
130             throw translateConstraintViolation(appointment, e);
131         } catch (SQLException e) {
132             if (e.getErrorCode() == DUPLICATE_ENTRY) {
133                 throw translateConstraintViolation(appointment, e);
134             }
135             throw new RuntimeException("Failed to save appointment", e);
136         }
137     }
138
139     /**
140      * The service layer already checked {@code existsActiveBooking} before
141      * calling this method, but that check can be overtaken by a concurrent
142      * booking that commits in between -- the {@code uk_dentist_slot} UNIQUE
143      * constraint is the actual guarantee. This turns its violation back into
144      * the same exception the caller already handles, so correctness never
145      * depends on the timing of two requests.
146      */
147     private SlotUnavailableException translateConstraintViolation(Appointment a, SQLException e) {
148         return new SlotUnavailableException(
149                 a.getDentist().getName() + " was booked for " + a.getAppointmentTime()
150                         + " on " + a.getAppointmentDate() + " by another user moments ago");
151     }
```

Figure 21: JdbcAppointmentDao.java, lines 106–151 — the INSERT, and translateConstraintViolation() turning a uk_dentist_slot violation back into SlotUnavailableException

AppointmentApiServlet — a complete web-service servlet (referenced in Section B.1)

AppointmentApiServlet.java

```java
package lk.icbt.clinic.servlet;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lk.icbt.clinic.dto.AppointmentResponse;
import lk.icbt.clinic.dto.CreateAppointmentRequest;
import lk.icbt.clinic.exception.InvalidBookingException;
import lk.icbt.clinic.model.Appointment;
import lk.icbt.clinic.service.BookingRequest;

import java.time.LocalDate;
import java.util.List;

/**
 * The appointment web service: {@code /api/appointments} and
 * {@code /api/appointments/{no}[/cancel]}.
 * <p>
 * One servlet handles all three shapes by branching on {@code pathInfo},
 * because the Jakarta Servlet API -- unlike Spring MVC's
 * {@code @GetMapping("/{no}")} -- has no built-in path-variable extraction.
 * Writing that extraction by hand is exactly what {@link JsonServlet#pathParam}
 * exists for.
 */
@WebServlet("/api/appointments/*")
public class AppointmentApiServlet extends JsonServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) {
        handle(resp, () -> {
            String pathInfo = req.getPathInfo();
            if (pathInfo != null && pathInfo.endsWith("/cancel")) {
                String appointmentNo = pathInfo.substring(1, pathInfo.length() - "/cancel".length());
                Appointment cancelled = ctx().appointmentService.cancel(appointmentNo);
                writeJson(resp, 200, new AppointmentResponse(cancelled));
                return;
            }

            Long staffId = staffIdHeader(req);
            CreateAppointmentRequest body = readBody(req, CreateAppointmentRequest.class);
            BookingRequest booking = new BookingRequest(
                    body.patientName, body.address, body.contactNumber,
                    body.dentistId, body.treatmentTypeId, body.appointmentDate, body.appointmentTime);

            Appointment booked = ctx().appointmentService.book(booking, staffId);
            writeJson(resp, 201, new AppointmentResponse(booked));
        });
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
        handle(resp, () -> {
            String pathInfo = req.getPathInfo();

            if (pathInfo == null || pathInfo.equals("/")) {
                // GET /api/appointments?date=2026-09-03
                String dateParam = req.getParameter("date");
                if (dateParam == null) {
                    throw new InvalidBookingException("date query parameter is required");
                }
                List<AppointmentResponse> list = ctx().appointmentService.findByDate(LocalDate.parse(dateParam))
                        .stream().map(AppointmentResponse::new).toList();
                writeJson(resp, 200, list);
                return;
            }

            // GET /api/appointments/{no}
            String appointmentNo = pathParam(req);
            Appointment appointment = ctx().appointmentService.findByAppointmentNo(appointmentNo);
            writeJson(resp, 200, new AppointmentResponse(appointment));
        });
    }
}
```

Figure 22: AppointmentApiServlet.java in full — doPost and doGet handling all three /api/appointments endpoint shapes by branching on pathInfo